Le board Linear se met à jour quand tu commites.

Un workflow n8n lit tes commits, retrouve le ticket correspondant et le met à jour. Ce récap reprend les notions de la présentation, l'installation complète et la démo. Il accompagne les slides.

```
$ git-sync
→ 2 commit(s) sur feat/PAR-5-books-crud (commits non pousses)
Ticket PAR-5 → In Progress (2 commit(s))
https://linear.app/equipe/issue/PAR-5
```

Backlog → In Progress → Done

Le ticket avance de lui-même : un commit le passe en In Progress, un commit `(closes-all)` avec des tests réussis le passe en Done.

Le problème et l'idée

Le board Linear ne reflète presque jamais la réalité. Un dev commence une tâche, le ticket ne bouge pas (ou n'existe pas encore), et le chef de projet n'a aucune visibilité sur ce qui avance.

> **Ce n'est pas un problème de discipline, c'est un problème d'incitation : celui qui déplace la carte n'en tire aucun bénéfice.**

Le seul geste demandé au dev est celui qu'il fait déjà : commiter. Le reste est automatique.

Ce que fait le système

Action du dev	Effet dans Linear
`git-sync` sur une branche qui contient l'identifiant (`feat/PAR-5-...`)	Le ticket PAR-5 passe en **In Progress** et reçoit un commentaire listant les commits. Aucune IA n'intervient.
Plusieurs `git-sync` sur le même ticket	Un commentaire s'ajoute à chaque fois. Le statut ne bouge pas : il ne change que si le dev le décide.
Un commit contenant `(closes-all)`	Les tests tournent en local. S'ils passent tous, le ticket passe en **Done** avec un récap des vérifications. Sinon il reste en In Progress.
Aucun identifiant dans la branche (chemin optionnel)	Claude lit les tickets ouverts et propose le plus proche. Au-dessus de 0,7 de confiance, le ticket est utilisé. Sinon un nouveau ticket est créé.

Tout est réversible : le workflow ne fait que changer un statut et ajouter un commentaire, rien de destructif.

Les notions à connaître

n8n et workflow

n8n est un outil d'automatisation. Un workflow est une suite de nœuds reliés par des flèches, qui s'exécute quand un événement le déclenche. Quatre types de nœuds suffisent ici :

- **Déclencheur** : démarre le workflow (le webhook).
- **Logique (IF)** : choisit un chemin (« référence trouvée ? »).
- **Code** : calcule ou analyse (« analyser le contexte »).
- **Action** : appelle une API (Linear, Anthropic).

Webhook

Une adresse HTTP qui attend des données. `git-sync` envoie un POST à `http://localhost:5678/webhook/git-sync` ; n8n reconnaît ce chemin et lance une exécution du workflow.

Architecture de n8n

Trois briques : le moteur qui exécute le workflow nœud par nœud, les nœuds qui font le travail, et le serveur HTTP qui reçoit les appels. Le workflow est un fichier JSON (un tableau `nodes`, un objet `connections`), exactement ce que le moteur exécute. On peut donc le versionner dans git.

Postgres

n8n y stocke ses workflows, ses identifiants chiffrés et l'historique des exécutions. Le nom `postgres` est du DNS interne Docker standard. La démo `demo-book-crud` n'utilise pas ce Postgres, ses données sont en mémoire.

États d'un nœud

Actif ou désactivé est un choix de configuration. Succès, erreur ou non atteint est le résultat d'une exécution. L'onglet **Executions** sert de diagnostic : on y voit les données qui ont traversé chaque nœud.

Seuil de confiance

Quand Claude rapproche un commit d'un ticket, il renvoie un score entre 0 et 1. En dessous de 0,7, rien n'est modifié sur un ticket existant.

Double verrou du Done

Un ticket ne passe en Done que si les deux conditions sont réunies :

1. le dev a écrit volontairement `closes-all` dans un commit ;
2. une vraie commande de test, exécutée en local, renvoie des vérifications toutes réussies.

n8n lit un résultat pass/fail réel. Il ne devine jamais.

Le parcours complet, du terminal à Linear

Les cercles pleins se passent sur ta machine, avant tout appel réseau. Les cercles vides se passent dans n8n ou chez Linear.

1. **Le dev tape `git-sync`** `local`
   À la racine du projet.

2. **Le script lit l'état du dépôt** `local`
   Branche, commits pas encore envoyés (marqueur dans `.git/n8n-last-sync`), fichiers modifiés, clé d'équipe (fichier `.n8nsync`).

3. **Si un commit contient `closes-all`, les tests tournent** `local`
   La commande de `.n8ntest` s'exécute sur la machine et le résultat réel de chaque vérification est récupéré.

4. **Le script envoie un JSON à n8n** `local`
   POST vers `localhost:5678/webhook/git-sync`, le conteneur Docker qui tourne sur la machine.

5. **n8n déclenche le workflow** `n8n`
   Un nœud Code cherche un identifiant du type `PAR-5` dans la branche et les messages de commit.

6. **Le ticket est identifié** `n8n`
   Trouvé : n8n interroge l'API GraphQL de Linear. Non trouvé : le chemin optionnel passe par Claude (rapprochement ou création).

7. **Linear est mis à jour** `Linear`
   Passage en In Progress, puis commentaire listant les commits.

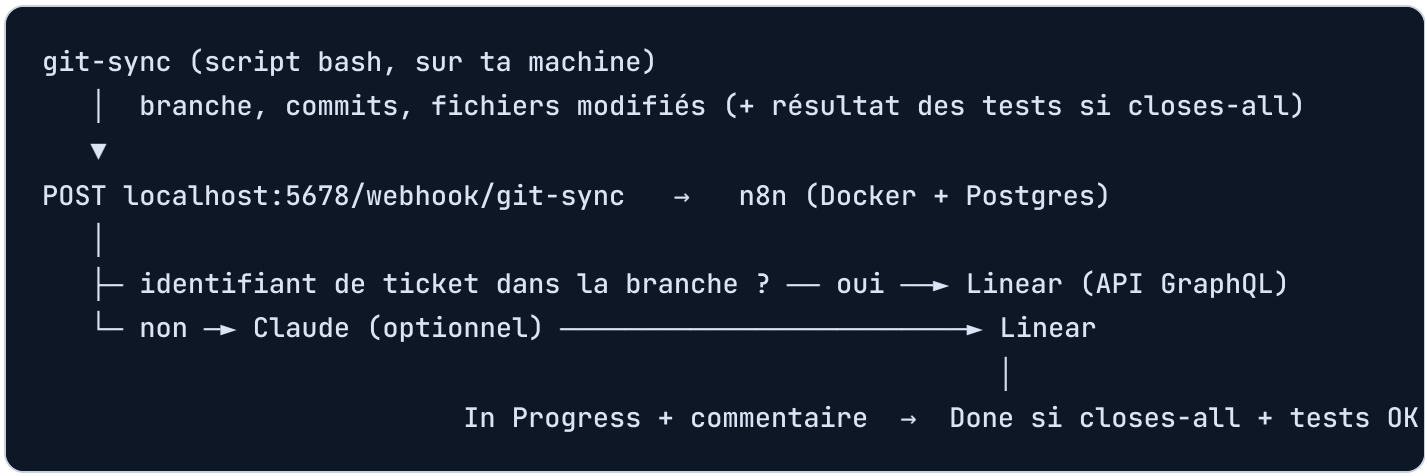
8. **Passage en Done si les deux verrous sont réunis** `Linear`
   Une branche parallèle passe le ticket en Done avec un commentaire récapitulatif des vérifications.

9 **Le marqueur local est mis à jour** `local`

Seulement si n8n a répondu 200. Un échec ne perd aucun commit, et relancer `git-sync` ne crée pas de commentaire en double.

> Le terminal affiche toujours « -> In Progress », même quand le ticket finit en Done juste après : la réponse HTTP part avant la fin de l'évaluation. Le vrai résultat est dans Linear.

Architecture



- **Tout est local.** n8n tourne en Docker sur la machine, aucun port exposé sur Internet, aucun tunnel.
- **Les tests tournent avant l'envoi.** n8n ne lit jamais le code, seulement un résultat pass/fail.
- **Deux appels sortent de la machine :** Linear et, en option, Anthropic. Claude ne voit que des noms de fichiers et des messages de commit.

Les 21 nœuds du workflow

Nœuds	Rôle
1 à 3	Webhook, analyse du contexte, test « référence trouvée ? »
4	Chemin direct : récupérer le ticket nommé dans la branche
5 à 10	Chemin IA (optionnel) : contexte équipe, appel à Claude, décision, création de ticket si besoin
11	« Normaliser le ticket » : les deux chemins convergent
12 à 14	Passage en In Progress, ajout du commentaire
15	Réponse HTTP au script
16 à 21	Mécanisme Done : mot-clé, décision sur les tests réels, passage en Done, commentaire de récap

Le détail de chaque nœud est dans `docs/03-workflow.md`.

Installation

Prérequis : Docker, `jq`, un compte Linear. La clé API Anthropic est optionnelle, elle ne sert qu'au rapprochement par IA. Le reste (In Progress, commentaires, Done via tests réels) fonctionne sans.

1 Installer n8n en local

10 minutes · `docs/01-installation.md`

```
cd docker
cp .env.example .env

# génère les deux secrets, puis remplace les REMPLACE_MOI dans .env
echo "POSTGRES_PASSWORD=$(openssl rand -hex 16)"
echo "N8N_ENCRYPTION_KEY=$(openssl rand -hex 32)"

docker compose up -d
docker compose ps    # n8n et postgres : running, postgres (healthy)
```

Ouvre http://localhost:5678 et crée le compte propriétaire. Il est local à ta machine, sans lien avec n8n Cloud.

> **La clé `N8N_ENCRYPTION_KEY` est le point critique.** n8n s'en sert pour chiffrer tes identifiants. Perdue ou modifiée après le premier démarrage, elle les rend illisibles, même dans une sauvegarde. Garde-la dans un gestionnaire de mots de passe. Le fichier `.env` n'est jamais versionné.

2 Créer les identifiants

10 minutes · `docs/02-credentials.md`

- **Linear** : Settings → Security & access → Personal API keys. Dans n8n : Credentials → type *Linear API*, nom `Linear`, authentification `API Token`, puis Test.
- **Anthropic (optionnel)** : crée une clé sur la console Anthropic. Dans n8n : Credentials → *Header Auth*, nom `Anthropic`, Name `x-api-key`, Value = la clé `sk-ant-...`.
- Note la **clé d'équipe** Linear : le préfixe de tes tickets (`PAR` dans `PAR-5`).

3 Mettre en place le workflow

5 minutes en important, 1 h 30 en le construisant · `docs/03-workflow.md`

Le plus rapide : importer `workflows/git-linear-sync.json` (menu `⋯` en haut à droite → Import from File), puis rattacher le credential Linear à chaque nœud Linear (plus aucun

triangle rouge). Active ensuite le workflow avec l'interrupteur **Active**.

4 Installer le script `git-sync`

10 minutes · `docs/04-script-git-sync.md`

```
ln -sf <chemin-du-depot>/bin/git-sync /usr/local/bin/git-sync
# Apple Silicon, si /usr/local/bin n'est pas dans le PATH :
# ln -sf <chemin-du-depot>/bin/git-sync /opt/homebrew/bin/git-sync

which git-sync && git-sync --help

# dans le dépôt du projet
echo "PAR" > .n8nsync
```

Commande	Effet
`git-sync`	Usage normal
`git-sync --dry-run`	Affiche ce qui serait envoyé, sans appeler n8n
`git-sync --all`	Ignore le marqueur et renvoie toute la branche
`git-sync --team WEB`	Force la clé d'équipe

5 Activer le passage en Done (optionnel)

Crée un fichier `.n8ntest` à la racine du projet, avec une seule ligne : la commande de test.

```
echo "cd demo-book-crud && npm test" > .n8ntest
```

Cette commande doit afficher, sur la dernière ligne de sa sortie, un JSON de cette forme :

```
{"allPassed": true, "results": [{"name": "...", "passed": true}]}
```

6 Valider avant de présenter

30 minutes · `docs/05-tests.md`

Cinq scénarios : ticket nommé dans la branche, rapprochement par IA, création de ticket, idempotence (relancer `git-sync` ne renvoie rien), passage en Done. Le document contient aussi un tableau de diagnostic des erreurs fréquentes.

Commandes Docker utiles

Besoin	Commande (depuis `docker/`)
Démarrer / arrêter	`docker compose up -d` / `docker compose stop`
Voir les logs	`docker compose logs -f n8n`
Mettre à jour n8n	`docker compose pull && docker compose up -d`
Sauvegarder la base	`docker compose exec postgres pg_dump -U n8n n8n > ~/n8n-backup.sql`

`docker compose down` supprime les conteneurs mais garde les volumes, donc tes workflows survivent. `docker compose down -v` efface tout.

La démo

Le projet `demo-book-crud`

Une petite API REST en TypeScript avec Hono, au stockage en mémoire. Son but est de produire de vrais commits à synchroniser, pas de livrer un service de production. Chaque

ressource correspond à un ticket Linear :

Ticket	Ressource
PAR-5	Livres (`/books`)
PAR-6	Auteurs (`/authors`)
PAR-7	Avis sur les livres (`/reviews`)
PAR-8	Health-check (`/health`)
PAR-9	Emprunts (`/loans`)

```
cd demo-book-crud
npm install
npm start            # lance l'API
npm test             # toutes les vérifications
npm run test:books   # une ressource à la fois
```

Chaque test correspond à un critère d'acceptation d'un ticket. Son résultat réel (pass/fail, jamais une estimation) est lu par `git-sync` via `.n8ntest` pour décider du passage en Done.

Préparation avant de démarrer

- `docker compose ps` : n8n et postgres tournent.
- Le workflow est Published, tous les nœuds Linear ont leur credential.
- Les tickets de démo (par exemple PAR-8 et PAR-9) sont en **Backlog** dans Linear.
- Trois onglets ouverts : n8n, Linear (vue board), terminal.
- `unset N8N_WEBHOOK_URL` pour repasser sur l'URL de production.
- `./setup-demo-book-crud.sh` lancé une fois à la racine du dépôt, juste avant de commencer : il recrée les fichiers de démo non committés.
- Branches et commits préparés à l'avance, rien n'est tapé en direct.

Déroulé

1. **La douleur**
   On ouvre le board Linear et on demande s'il reflète ce sur quoi l'équipe travaille. On laisse le silence répondre.

2. **La promesse**
   Le board se met à jour tout seul à partir de la seule chose qu'un dev fait toujours : commiter.

3. **Démo 1 : le cas simple**
   Branche qui nomme le ticket. `git log --oneline -3` puis `git-sync`. On rafraîchit Linear : le ticket est en In Progress, avec le commentaire des commits. Aucune IA, n8n a lu le nom de la branche.

4. **Démo 2 : le suivi continu**
   Deux ou trois commits sur le même ticket, chacun suivi d'un `git-sync`. Les commentaires s'accumulent, le statut reste en In Progress.

5. **Démo 3 : le passage en Done**
   On montre le commit `(closes-all)`, puis `git-sync`. Les tests tournent en local avant l'envoi. On rafraîchit Linear : le ticket est en Done, avec le récap des vérifications.

6. **n8n en deux mots**
   Le workflow dézoomé (21 nœuds, aucune ligne de code déployée) et l'onglet Executions.

7. **Ce que ça coûte, puis clôture**
   Voir la section suivante.

Le déroulé exact et le pense-bête de commandes sont dans `presentation/notes-presentateur.html`.

Coûts et licence

Poste	Coût
n8n auto-hébergé	Gratuit et illimité en usage interne (licence *fair-code*, Sustainable Use License). La revente de n8n en tant que service est interdite.
Linear	Un compte suffit.
API Anthropic	Optionnelle. De l'ordre du centime par rapprochement, et seulement quand la branche ne nomme pas déjà le ticket. Sinon, zéro appel IA.

Aller plus loin

- **Serveur partagé.** Pour que toute l'équipe en profite, n8n peut passer sur un VPS. La procédure et la configuration de sécurité sont dans `docs/07-vps.md`.
- **Hook `pre-push`.** Il déclenche `git-sync` à chaque push. Le `|| true` est volontaire : une panne de n8n ne doit jamais bloquer un push.
- **Maintenance.** Un workflow non maintenu devient une dette. Versionner son export (`workflows/git-linear-sync.json`) évite qu'il ne vive que sur un portable.

Contenu du dépôt

```
docker/docker-compose.yml        n8n + postgres
docker/.env.example              secrets à copier en .env
bin/git-sync                     le script CLI
workflows/git-linear-sync.json   export du workflow
docs/                            le pas-à-pas (01 à 07)
.n8nsync                         clé d'équipe Linear pour ce dépôt
.n8ntest                         commande de test pour le passage en Done
demo-book-crud/                  API de démonstration (Hono + TypeScript)
setup-demo-book-crud.sh          régénère demo-book-crud/
presentation/                    slides + notes présentateur
```